

IS216 Mini Lab Test

[25 marks]

General Instructions:

- This is a time-bound (1 hour 30 minutes), open-book, open-Internet, and **individual** test.
- You must test your web pages using **Google Chrome Web Browser** Version 103.0.x or later). Your graders will be using only **Google Chrome Web Browser** (Version 103.0.x or later) to test your web pages.
- **No questions will be entertained** by the IS216 teaching team (faculty/instructor/Teaching Assistants) during the test period. If necessary, make your own assumptions and proceed to complete test questions.
- You must use Vue 3 (Composition API with <script setup>), standard HTML5, CSS, Bootstrap (Version 5.3), JavaScript and Axios in your solutions unless the question specifies otherwise. Do not use other third-party front-end frameworks or libraries unless explicitly permitted.
- Use of innerHTML/v-html is prohibited. Any occurrence of innerHTML/v-html found in your code will result in a penalty of 1 mark per occurrence.
- Use meaningful names for HTML class/id, JavaScript variables/functions, and Vue reactive state. You must indent your Vue/HTML/CSS/JavaScript code properly. Failure to do so will attract a penalty of up to 20% of your score for the corresponding question.
- You **MUST** include your name as author in the comments of all your submitted source files. Failure to do so will attract a penalty of up to **20%** of your score for the corresponding question. For example, if your registered name is "TAN So Tong" and email ID is tan.sotong.2023, include the following comment at the beginning of each source file you write.

HTML files	CSS, JavaScript files
<pre><!-- Name: TAN So Tong Email: tan.sotong.2023 --></pre>	<pre>/* Name: TAN So Tong Email: tan.sotong.2023 */</pre>

- You may wish to comment out the parts in your code which cause errors. Commented code will not be marked.

Academic Integrity

- All student submissions will be thoroughly checked by an SMU-approved source code plagiarism checker software program AND an additional external software program. The source code checking will be conducted across all submissions from all sections of IS216.
- Suspected plagiarism cases will be reported immediately to the IS216 faculty in charge and SCIS Dean's Office for further investigation.

- Students in the suspected cases will be informed accordingly by their section faculty, and the incident will be escalated to the SMU University Council of Student Conduct.
- More information about the SMU Student Code of Conduct can be found at this link: <https://smu.sharepoint.com/sites/ucsc/Documents/Code%20of%20Academic%20Integrity.pdf>

Submission Instructions

- Due Date**
 - 4 October 2024 (Friday) 17:15 PM Singapore Time**
 - Late submission policy is as follows:

Submit within 5 minutes of set deadline	10% penalty of your entire test's score
Beyond 5 minutes	0 mark

- Zip** up all files in Q1/Q2/Q3/Q4 folders into <YOUR_SMU_ID>.zip
 - For example, **tan.sotong.2023.zip**
 - Verify by unzipping this zip file – check the content inside**
 - Incorrect submission file name** WILL attract a penalty of up to **20%** of your score for the entire test.
- Only **zip** format is accepted.
 - .7z, rar** or other **compression formats** are **NOT** accepted.
 - Until the correct **zip** format is submitted again by the student, it will be assumed that the student has NOT made the submission and late submission policy will apply.
- Submit the **zip** file to the following location:
 - IS216-MERGED eLearn** page: <https://elearn.smu.edu.sg/d2l/home/398535>
 - Go to **Assignments** → **Mini Lab Test** → **Submit your ZIP file**
 - It is your (student's) individual responsibility to ensure that the zip file submission was successful.**
 - Your section faculty and Teaching Assistants will NOT verify the submission for you.**

Legend



Do NOT edit this given resource file.



Your answer/code goes here into this given resource file.

Q1. Basic CSS

[9 Marks]

Given resources in folder q1

 q1.vue

 images/book_cover_image.png

 q1.css

Instructions

1. You are to style the HTML elements using **vanilla CSS** (no Bootstrap).
2. Open *q1.vue* in a browser and examine how it looks.
3. You are to edit only *q1.css* file.

Note: You are to define CSS styles in the labelled locations e.g. /* part A */ within the css file to match the screenshots to the best of your interpretation **where the requirement is not stated explicitly**. This includes margin, padding, font style, font size, font weight, etc. **DO NOT** remove the existing styles or add new selectors in the css file.

Part A: Style the header as shown below. (1.5 marks)



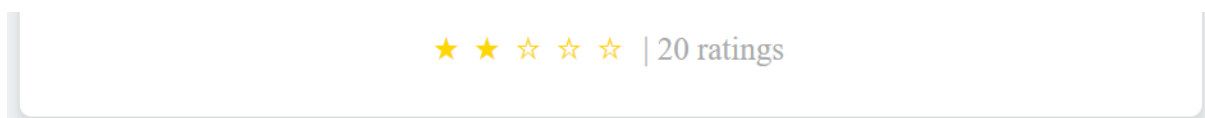
- Set the size of the text as 40 pixels.
- Centralize the text.
- Set the color of the text as white or #fff in hexadecimal.
- Set the background color as #3498db in hexadecimal.

Part B: Style the article section with the class of “book-review” as shown below. (3 marks)



- Set the background color to white or #fff in hexadecimal.
- Insert a solid lined border that is 1 pixel thick with color #ddd in hexadecimal.
- Set the radius of the border to 8 pixels.
- Centralize the contents (including the book image) within the article section.
- Set the spacing between the top border and contents contained within the article section and the spacing between the bottom border and contents contained within the article section to be both 30 pixels.
- Change the width of the book image to 200 pixels.

Part C: Style the rating section as shown below. (3 marks)



- Set the color of the stars as gold.
- Set size of the star as 20 pixels.
- The stars are displayed in a single row (with no bullet points).
- The stars and the text “20 ratings” are displayed in a single row (with the given padding applied).

Part D: Style the footer as shown below. (1.5 marks)

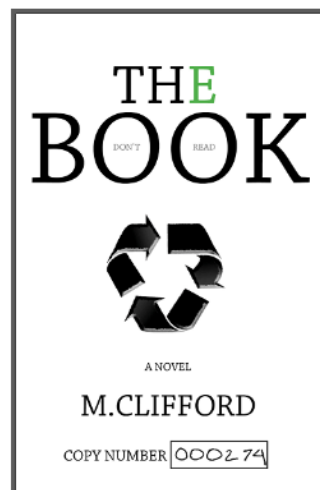
© 2024 Literary Corner. All rights reserved.

- Set the color of the text as white or #fff in hexadecimal.
- Set the background color as #2c3e50 in hexadecimal.
- Centralize the footer text by inserting an adequate CSS style within the “footer p” selector (This is the only accepted way of fulfilling this requirement).
- Do not override the existing width property.

The final webpage should look like the output shown in the next page:

Literary Corner

Featured Review



by M. Clifford

The book that does not deserved to be read

★ ★ ☆ ☆ ☆ | 20 ratings

© 2024 Literary Corner. All rights reserved.

Q2. Bootstrap: Debug Layout

[5 Marks]

Given resources in folder q2



q2.vue



joyboy.jpg



q2_readonly.css

Instructions

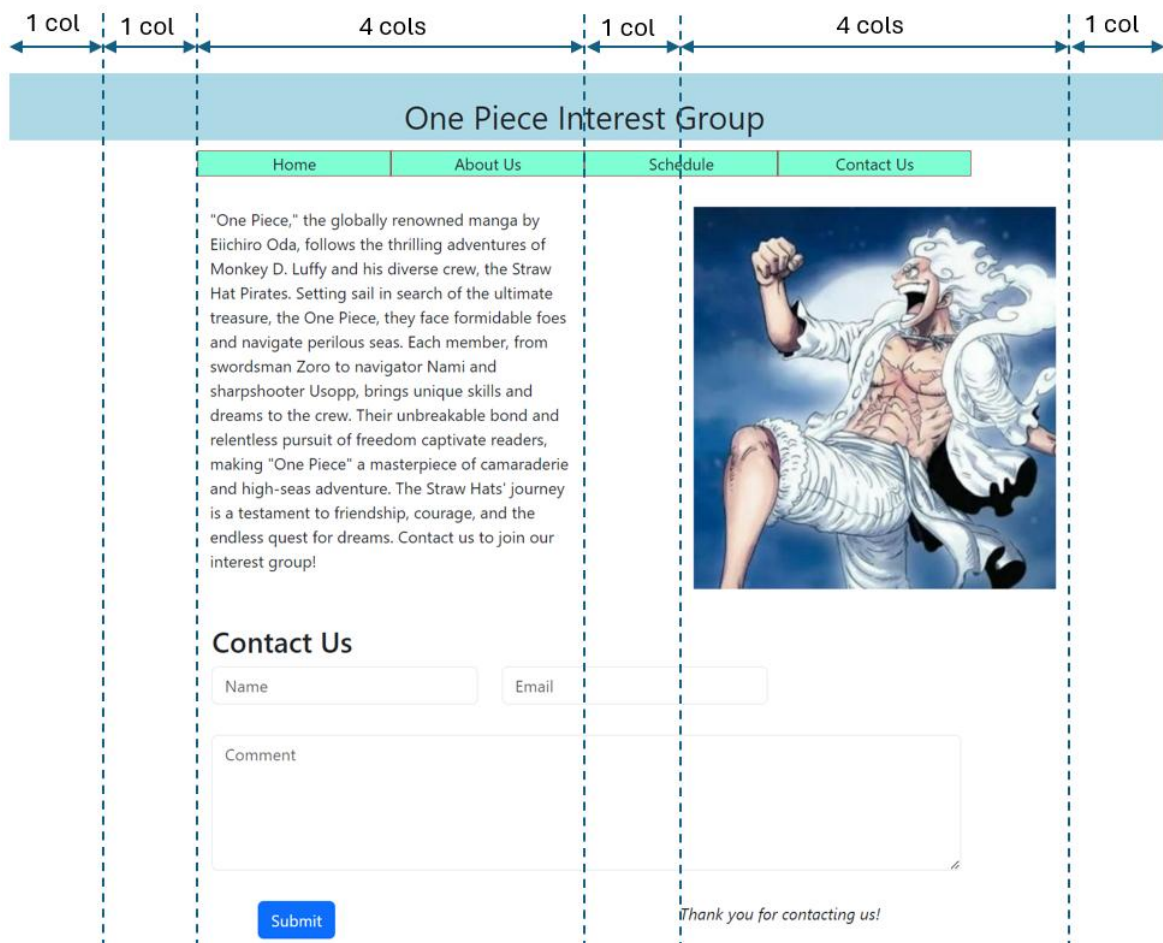
1. You are to solve the web page layout problem using Bootstrap's *grid and breakpoints*.
2. Open `q2.vue` in a browser and examine how it looks when you change the browser width.
3. You should notice that the given code has already set grid layout and breakpoints for different browser widths, but the code is buggy.

Note:

- Debug and add code to achieve the web page layout at different browser widths specified below.
- You are not required to add/edit other styles, such as margin, padding, font style, font size, etc., not mentioned in the requirements.

Debug and add code in `q2.vue` so that it behaves as follows:

Part A: When the browser width is ≥ 992 px (lg breakpoint), the form should appear as shown below. (1 mark)



- The div with `id="desc"` (the paragraph of text) and the div with `id="image"` (the image) should occupy **4 columns** of Bootstrap 12-column grid, each.

Part B: When the browser width is ≥ 768 px (md breakpoint) but < 992 px (lg breakpoint), the “Contact Us” portion of the form should appear as shown below. (2 marks)

The diagram illustrates a Bootstrap 12-column grid layout for a "Contact Us" form in the medium (md) breakpoint. The grid is defined by vertical dashed lines and horizontal arrows indicating column counts. The layout is as follows:

- Header:** "Contact Us" is centered in the first 5 columns.
- Form Fields:**
 - Name:** A text input field spanning 5 columns.
 - Email:** A text input field spanning 5 columns.
 - Comment:** A large text area spanning the full 10 columns (5 + 5).
- Footer:**
 - The text "Thank you for contacting us!" is centered across the 10 columns.
 - The "Submit" button is centered across the 10 columns.

Column counts are indicated by arrows at the top: 1 col (left margin), 5 cols (Name and Email), 5 cols (Comment), and 1 col (right margin). A total of 12 cols is indicated at the bottom.

- Name and Email fields occupy **5 columns** of Bootstrap 12-column grid, each.
- The “Thank you for contacting us!” text **comes before** the Submit button. This requirement applies even when the browser width is < 768 px.

Part C: When the browser width is < 768 px (md breakpoint), the form should appear as (partially) shown below. (2 marks)

1 col 5 cols 5 cols 1 col

One Piece Interest Group

Home About Us

Schedule Contact Us

"One Piece," the globally renowned manga by Eiichiro Oda, follows the thrilling adventures of Monkey D. Luffy and his

⋮

Contact Us

Name

Email

Comment

Thank you for contacting us!

Submit

1 col 10 cols 1 col

- The menu items of "Home" and "About Us" are side-by-side and occupy **5 columns** of Bootstrap 12-column grid, each.
- The menu items of "Schedule" and "Contact Us" are side-by-side and occupy **5 columns** of Bootstrap 12-column grid, each.
- The Name field occupies **10 columns** of Bootstrap 12-column grid.
- The Email field occupies **10 columns** of Bootstrap 12-column grid.
- The "Thank you for contacting us!" text still **comes before** the Submit button.

Q3. Vue Reactivity & Event Handling

[6 Marks]

Given resources in folder q3

 q3.vue

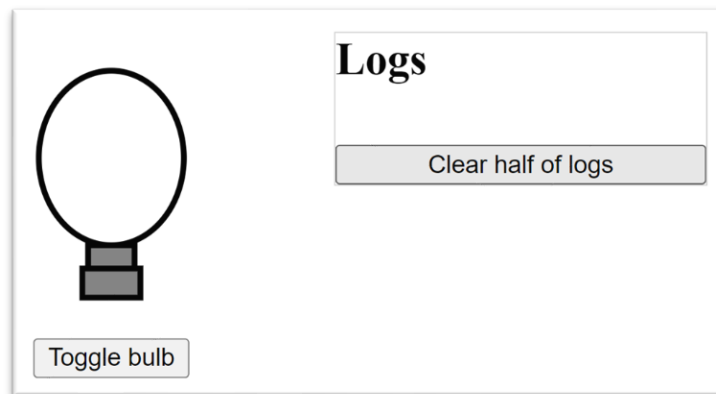
 q3_readonly.css

Instructions

1. You are to edit ONLY q3.vue.
2. The template in q3.vue is already set up so that clicking the “Toggle bulb” button calls `changeColor()`, while clicking the “Clear half of logs” button calls `halveLogs()`. You need not add the event handlers for these two buttons.
3. This question will be marked by an automated testing script.
4. Debug and add code in the `<script setup>` section of q3.vue so that the component behaves as follows.

Original output

When the q3 component is displayed for the first time, it should show the following output:



Tasks to complete

Part A: Change color of bulb (1 mark)

Add code in `changeColor()` so that when the user clicks the “Toggle bulb” button, the reactive variable `bulbColor` toggles between the strings "white" and "yellow". The template is already bound to `bulbColor`, so changing this reactive state should update the bulb colour automatically.

Note: If you are successful, when you click the “Toggle bulb” button, the bulb changes color with a transition time of 1 second. This transition time is dictated by `q3_readonly.css`, which you do NOT need to change.

Part B: Delay use of “Toggle bulb” button (1 mark)

Add code in the `delayButton()` function so that when the user clicks on the “Toggle bulb” button, the “Toggle bulb” button will be disabled for 1 second, before it is enabled for use again.

Hint: `setTimeout(callbackFunc, delayInMilliseconds)` calls a function after a number of milliseconds.

Sample code:

```

const delay = 1000 // milliseconds => 1 second
function callback() {
  console.log(`This function is invoked after ${delay}ms`)
}

console.log(`Before setTimeout(): ${new Date()}`)
setTimeout( callback, delay)
console.log(`After setTimeout(): ${new Date()}`)

```

Console:

```

Before setTimeout(): Mon Sep 28 2026 20:02:39 GMT+0800 (Singapore Standard Time)
After setTimeout(): Mon Sep 28 2026 20:02:39 GMT+0800 (Singapore Standard Time)
This function is invoked after 1000ms

```

For addLog() function

The addLog() function is supposed to add logs of user interaction to the reactive logs array, which is rendered as an ordered list using Vue.

- If the user pressed the “Toggle bulb” button to turn the fill of the bulb **yellow**, the three logs to add should be:

"User interacts.", "ON.", "Bulb lights up."

- If the user pressed the “Toggle bulb” button to turn the fill of the bulb **white**, the three logs to add should be:

"User interacts.", "OFF.", "Bulb turns off."

Part C: Add code to addLog() function (1 mark)

Add code to addLog() so that, for each element of the newLog array passed in, the string is added to the reactive logs array. The template will automatically render the updated logs using v-for.

Example: After a page refresh, when the “Toggle bulb” button is clicked 3 times, the following logs should be shown:

Logs

1. User interacts.
2. ON.
3. Bulb lights up.
4. User interacts.
5. OFF.
6. Bulb turns off.
7. User interacts.
8. ON.
9. Bulb lights up.

Clear half of logs

Debugging Task

Part D: Show errorMsg when there are too many logs (0.5 marks)

In `changeColor()`, debug the erroneous condition so that `errorMsg` is set to “Clear some logs before proceeding” when adding another set of 3 logs would potentially exceed the maximum number allowed under `maxLogs = 10`. Do not change the given style associated with the error message.

Example: After a page refresh, when the “Toggle bulb” button is clicked a 4th time, the following output should be shown:

Logs

1. User interacts.
2. ON.
3. Bulb lights up.
4. User interacts.
5. OFF.
6. Bulb turns off.
7. User interacts.
8. ON.
9. Bulb lights up.

Clear half of logs

*Clear some logs before
proceeding*

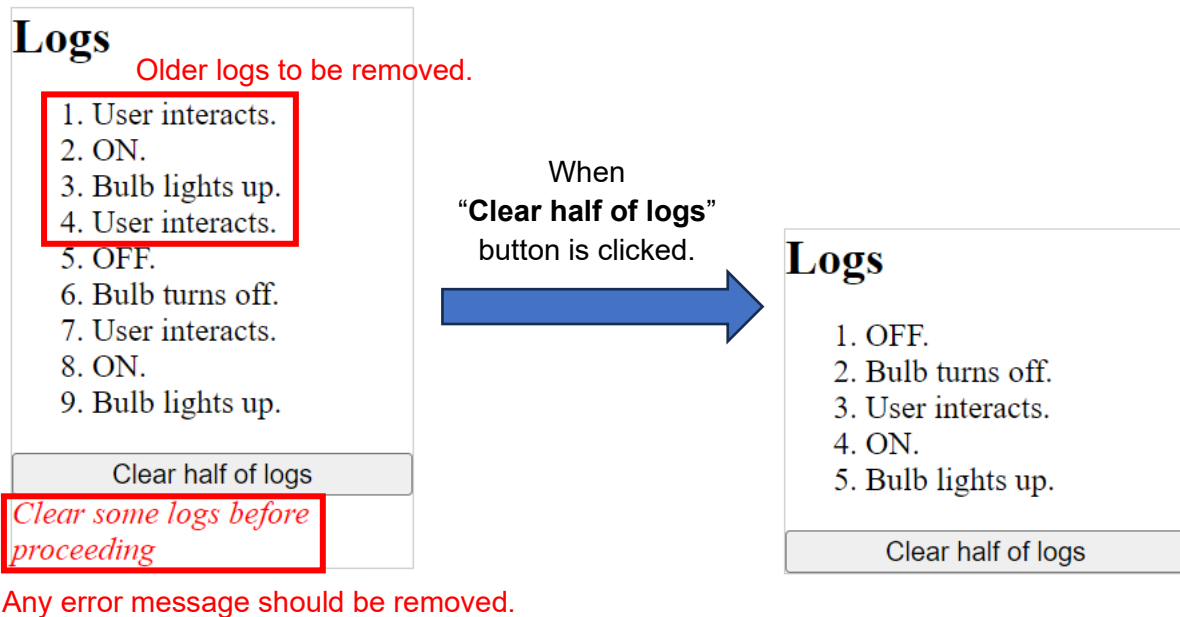
For `halveLogs()` function

The `halveLogs()` function is called when the user clicks on the “Clear half of logs” button.

Part E: Add code to `halveLogs()` function (2 marks)

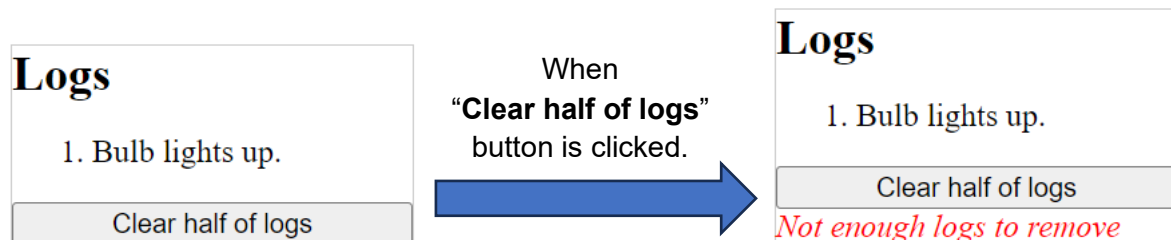
Requirement 1: If there is an odd number of existing logs, remove a whole number of oldest logs just less than half of the number of logs. If there was an error message, remove it. (1 mark)

Example: Original number of logs is 9. After the “Clear half of logs” button is pressed, 4 of the older logs (4 being the whole number just less than 4.5, half of 9) should be removed. Error message removed.



Requirement 2: If there is only 1 log left, do NOT remove the last remaining log. Instead, set the reactive errorMsg variable to “Not enough logs to remove”. Do not change the given style associated with the error message. (1 mark)

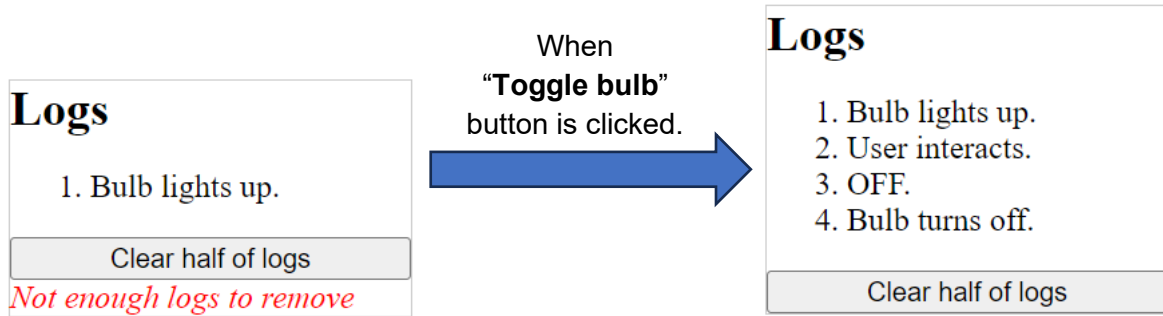
Example:



Part F: Remove any error message if “Toggle bulb” works i.e. logs can be added. (0.5 marks)

In any appropriate part of q3.vue, add code to clear errorMsg if “Toggle bulb” works, i.e. logs can be added again.

Example: The user tried to clear the logs, but as there was only 1 log left, an error message is shown. The user then clicks on the “Toggle bulb” button, which adds the necessary logs, and clears the error message.



Q4: Vue, AXIOS & JSON

[5 Marks]

Given resources in folder q4



q4.vue



q4_readonly.css



server/server.js



server/bike_stations.json

Instructions

1. Run the provided Node/Express `server.js` in the `server` folder for `q4` before testing this question, using `pnpm install`, then `pnpm dev`. The server provides the REST API used by the Vue component.
2. You are to only insert code into `q4.vue` to fulfil the following requirements.

Part A: (2 marks)

1. Complete the function "getStations". Inside the function, use an Axios GET request to the provided API endpoint (no parameter required) to retrieve the list of bike stations and their availability information. Store the returned station data in the provided reactive stations variable.
2. Use Vue template syntax to display the list of bike stations and the availability of bicycles and docks in each station.

3. The required station-card structure is already indicated in `q4.vue`. Render the station list dynamically using Vue directives/interpolation (for example, `v-for` and `{{ }}`). Do NOT manually create DOM elements or use `innerHTML/v-html`.

```
<!--  
  <div class="station-item">  
    <h3>Central Park Station</h3>  
    <p>Available Bikes: 10 </p>  
    <p>Available Docks: 3 </p>  
  </div>  
-->
```

4. The desired output is shown below.

City Bike Share

Central Park Station

Available Bikes: 11

Available Docks: 4

Downtown Plaza Station

Available Bikes: 5

Available Docks: 10

Riverside Park Station

Available Bikes: 8

Available Docks: 7

University Campus Station

Available Bikes: 3

Available Docks: 12

Shopping District Station

Available Bikes: 2

Available Docks: 2

Rent a Bike

Select a station



Rent Bike

Return a Bike

Select a station



Return Bike

Part B: (1 mark)

1. Use Vue template syntax to populate the station locations in the “Select a station” dropdown lists for both “Rent a Bike” and “Return a Bike”.
2. The station data should come from the Axios GET request in Part A. Dynamically render an `<option>` for each station using Vue (for example, `v-for`). Each option value must be bound to the corresponding station id. The selected values are already connected to Vue reactive state using `v-model`.

```
<!--
<option value="1">Central Park Station</option>
-->
```

3. The desired output is shown below.

Rent a Bike

Select a station ▼

Rent Bike

Return a Bike

Select a station ▼

Return Bike

Select a station

Central Park Station

Downtown Plaza Station

Riverside Park Station

University Campus Station

Shopping District Station

Part C: (1.5 marks)

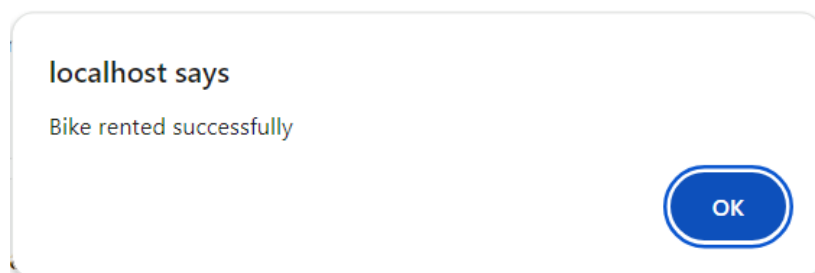
1. Complete the function `putAction()` to perform either a bike rental or bike return by making an Axios PUT request to the provided API endpoint:

`http://127.0.0.1:8000/api/:id`
 where `:id` is replaced by the ID of the selected station.

2. The function parameters `“action”` and `“stationId”` are provided. Use `stationId` as part of the request URL. The request body should contain:

`action`: the value of the action parameter
 The value of action will be either `'rent'` or `'return'`.

3. For a successful bike rental or return, display an alert box containing the message retrieved from the response object of the Axios **PUT** request. The message must **NOT be HARDCODED**.



4. Upon successful bike rental at a selected station (e.g. Riverside Park Station),

- “Available Bikes” should decrease by 1 and
- “Available Docks” should increase by 1.

Upon successful bike return,

- “Available Bikes” should increase by 1 and
- “Available Docks” should decrease by 1.

A sample output for successful bike rental is shown below.

Note: After a successful action, the displayed “Available Bikes” and “Available Docks” values should update without reloading the page. Use the provided Vue/Axios flow to refresh the reactive station data.

Before bike rental	After bike rental
Riverside Park Station Available Bikes: 8 Available Docks: 7	Riverside Park Station Available Bikes: 7 Available Docks: 8

Part D: (0.5 marks)

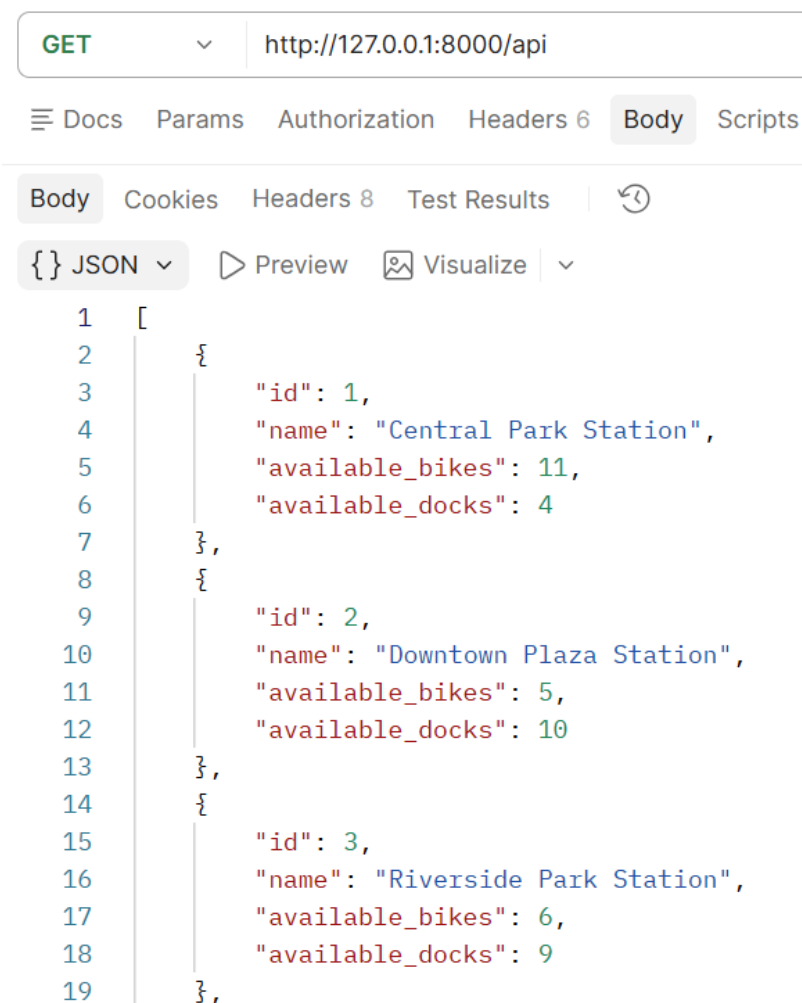
1. When there are no available bikes at a selected station and the user tries to rent a bike at that station, an alert box with the message “Error renting bike” should appear. **Hint:** Handle the HTTP 400 error returned by the Axios PUT request.

No available bikes at station	Error
Shopping District Station Available Bikes: 0 Available Docks: 4	localhost says Error renting bike OK

2. When there are no available docks at a selected station and the user tries to return a bike at that station, an alert box with the message “Error returning bike” should appear. **Hint:** Handle the HTTP 400 error returned by the Axios PUT request.

No available docks at station	Error
Shopping District Station Available Bikes: 4 Available Docks: 0	localhost says Error returning bike OK

Samples of the q4 API GET and POST requests using Postman:



PUT

▼

http://127.0.0.1:8000/api/2

☰ Docs

Params

Authorization

Headers 8

Body ●

Sc

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

```
1  {
2  |    "action": "rent"
3  }
```

Body

Cookies

Headers 8

Test Results

↺

{ } JSON ▼

▶ Preview

🖼 Visualize ▼

```
1  {
2  |    "message": "Bike rented successfully"
3  }
```

PUT

▼

http://127.0.0.1:8000/api/2

☰ Docs

Params

Authorization

Headers 8

Body ●

Script

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

```
1  {
2  |    "action": "return"
3  }
```

Body

Cookies

Headers 8

Test Results

↺

{ } JSON ▼

▶ Preview

🖼 Visualize ▼

```
1  {
2  |    "message": "Bike returned successfully"
3  }
```

End of Paper